

TD n°5 - Flottants et complexité

Exercice 1

Écrire les nombres suivants sous forme flottante, sur 32 bits. Préciser à chaque fois s'il s'agit d'une représentation exacte ou d'une représentation approchée.

1. 56.25

2. -30.15625

3. 145.758

Exercice 2

On vous donne les nombres suivants, sur 32 bits. Déterminer ce qu'ils représentent

1. 1|10010101|101000000000000000000000

2. 0|01110101|001001100100000000000000

3. 0|11111111|000000010000010000000000

4. 1|00000000|011000000000000000000000

Exercice 3

Vrai ou faux ?

1. Une complexité en $\Theta(n)$ est toujours considérée comme meilleure qu'une complexité en $\Theta(n^2)$.
2. Une boucle `for(int i=0; i<n; i+=1){ ... }` a une complexité en $\Theta(n)$.
3. La copie d'un tableau d'entiers de taille n en C prend un temps $\Theta(n)$.
4. Un $\Theta(n)$ est toujours un $O(n)$.
5. La taille en mémoire d'un entier n est l'entier n lui-même.
6. La complexité d'un programme Python est toujours plus élevée que celle du même programme en C.

Exercice 4

Montrer les comparaisons de suites suivantes en revenant à la définition de O et Θ :

- $\frac{n(n+1)}{2} = \Theta(n^2) (n \rightarrow +\infty)$
- $\log_2(n+1) + \ln(2n^2) = O(\ln(n)) (n \rightarrow +\infty)$

Exercice 5

Pour chacune des fonctions ci-dessous, donner le nombre d'additions effectuées en fonction des valeurs des arguments.

```
void f1(int n, int m){
    int i = 1;
    int j = 1;
    while(i<=n && j<=m){
        i = i+1;
        j = j+1;
    }
}
```

```
void f2(int n, int m){
    int i = 1;
    int j = 1;
    while(i<=n || j<=m){
        i = i+1;
        j = j+1;
    }
}
```

```
void f3(int n, int m){
    int i = 1;
    int j = 1;
    while(i<=n){
        if(j<=m){
            j = j+1;
        }
        i = i+1;
    }
}
```

```
void f4(int n, int m){
    int i = 1;
    int j = 1;
    while(i<=n){
        if(j<=m){
            j = j+1;
        }
        else{
            i = i+1;
            j = 1;
        }
    }
}
```

Exercice 6

Pour les suites u_n suivantes, trouver une suite v_n la plus simple possible telle que $u_n = \Theta(v_n)$. Vous pouvez justifier succinctement sans revenir à la définition de Θ si vous manipulez des suites classiques.

1. $u_n = 2^{35} + 5n^8 + 2076n^7$

2. $u_n = 1024n + 2048\sqrt{5n}$

3. $u_n = n^2/3 + 10^{100}n$

4. $u_n = 4\log(5n) + \sqrt{3n}$

5. $u_n = 2n \sum_{i=1}^{2n} (3i + 1) + \sum_{p=1}^n 2^p$